

## День 2. Інженерія ПЗ, частина 2

*Проектування, UML, патерни, реалізація, тестування та методології розробки*

### Проектування програмного забезпечення

Продовжуємо інженерію ПЗ. У першій частині ми визначили вимоги — зрозуміли, що будувати. Тепер переходимо до проєктування — це етап, на якому вирішують, як саме систему буде влаштовано, перш ніж писати код.

Розрізняють кілька видів проєктування. Структурне проєктування розбиває систему на функціональні блоки та визначає потоки даних між ними. Об'єктно-орієнтоване проєктування описує систему через об'єкти, що мають дані й поведінку. Функціональне проєктування зосереджується на функціях і перетвореннях даних. Архітектурне проєктування визначає загальну структуру системи на найвищому рівні: з яких великих компонентів вона складається і як вони взаємодіють. Інтерфейсне проєктування описує, як система взаємодіє з користувачем та з іншими системами.

Також розрізняють парадигми проєктування — це загальні підходи. Функціональна декомпозиція згори донизу: ми беремо велику задачу й розбиваємо її на дрібніші підзадачі, ті — ще дрібніші, і так до простих функцій. Архітектура, орієнтована на дані: у центрі стоїть структура даних, а функції будуються навколо неї. Об'єктно-орієнтований аналіз і проєктування: систему моделюють як набір взаємодіючих об'єктів. Подієво-керована архітектура: система реагує на події, наприклад натискання кнопки чи надходження повідомлення, і будується навколо обробки цих подій.

## UML та моделювання класів

Для проєктування об'єктних систем використовують мову UML — уніфіковану мову моделювання. Це набір стандартних діаграм для опису систем. На етапі проєктування спершу ідентифікують класи предметної області — тобто визначають, які об'єкти існують у задачі: студент, курс, оцінка. Потім будують діаграму класів, яка показує ці класи, їхні атрибути, методи та зв'язки й ієрархію між ними.

Окрім діаграми класів, для опису того, як саме реалізуються варіанти використання, застосовують ще дві діаграми. Діаграма послідовностей показує, у якому порядку об'єкти обмінюються повідомленнями в часі — згори вниз як хронологія. Діаграма комунікації показує те саме, але з акцентом на зв'язки між об'єктами, а не на час. Обидві описують один і той самий сценарій з різних боків.

## Патерни проєктування

Тепер важлива тема — патерни, тобто шаблони проєктування. Патерн — це типове, перевірене рішення поширеної задачі проєктування. Це не готовий код, а ідея, схема рішення. Класичні патерни поділяють на три групи, і це зручний спосіб їх запам'ятати.

Перша група — породжувальні патерни, що відповідають за створення об'єктів. До них належить абстрактна фабрика, англійською *Abstract Factory*: вона створює цілі сімейства пов'язаних об'єктів, не прив'язуючись до їхніх конкретних класів.

Друга група — структурні патерни, що відповідають за компонування об'єктів у більші структури. Сюди входять кілька важливих. Фасад, англійською *Facade*, надає спрощений єдиний інтерфейс до складної підсистеми, ховаючи її складність. Декоратор, англійською *Decorator*, динамічно додає об'єкту нову поведінку, обгортаючи його, без зміни

самого класу. Легковаговик, англійською Flyweight, економить пам'ять, спільно використовуючи однакові частини багатьох об'єктів замість їх дублювання. Замісник, англійською Proxy, підставляє замість справжнього об'єкта посередника, який контролює доступ до нього.

Третя група — поведінкові патерни, що відповідають за взаємодію та розподіл обов'язків між об'єктами. Спостерігач, англійською Observer, дозволяє одним об'єктам автоматично отримувати сповіщення про зміни в іншому об'єкті — це основа підписок і подій. Стратегія, англійською Strategy, дає змогу взаємозамінно підставляти різні алгоритми для однієї задачі. Відвідувач, англійською Visitor, дозволяє додавати нові операції до об'єктів, не змінюючи їхніх класів.

Ланцюжок обов'язків, англійською Chain of Responsibility, передає запит по черзі ланцюжком обробників, доки хтось його не опрацює.

І окремо стоїть патерн MVC — модель-вид-контролер. Це архітектурний патерн, який розділяє застосунок на три частини: модель — це дані й логіка, вид — це те, що бачить користувач, а контролер — це посередник, що обробляє дії користувача й оновлює модель та вид. Сенс MVC — відокремити дані від їх відображення.

Для іспиту не треба знати реалізацію кожного патерна. Треба впізнавати його за коротким описом призначення: фасад — спрощує доступ, спостерігач — сповіщає про зміни, стратегія — взаємозамінні алгоритми, замісник — контролює доступ, і так далі.

## **Реалізація програмного забезпечення**

Наступний етап — реалізація, тобто написання коду. Тут є кілька вимог і засобів, які варто знати на рівні понять. До оформлення коду висувають вимоги щодо стилю: код має бути читабельним, розбитим на логічні структуровані одиниці, а змінні, класи й об'єкти — мати

зрозумілі осмислені назви. Існують засоби автоматичної генерації коду, що створюють шаблонний код за моделлю чи описом.

Окрема важлива тема — налагодження, англійською *debugging*, тобто пошук і виправлення помилок. Основні інструменти: точки зупинки, англійською *breakpoints*, які зупиняють виконання програми в потрібному місці, щоб роздивитися її стан; спостереження за змінними, англійською *variable watch*, що показує поточні значення; виведення на консоль, тобто друк проміжних повідомлень; налагоджувач, англійською *debugger*, — спеціальна програма для покрокового виконання; і аналізатори коду, англійською *code analyzers*, які автоматично знаходять потенційні проблеми в коді ще до запуску.

Важливі також керування конфігурацією та версіями — це системи на кшталт систем контролю версій, що відстежують зміни в коді й дозволяють працювати команді. І сучасний підхід — постійна інтеграція та постійне впровадження, англійською *Continuous Integration* і *Continuous Delivery*, скорочено *CI/CD*. Це автоматизація: щойно розробник додає зміни, вони автоматично збираються, тестуються і за потреби розгортаються. Це зменшує помилки й пришвидшує випуск.

## **Забезпечення якості: тестування, верифікація, валідація**

Тепер дуже важлива тема, де часто плутаються в термінах. Розрізняють тестування, верифікацію та валідацію. Тестування — це процес виконання програми з метою знайти дефекти. Верифікація відповідає на питання «чи правильно ми будуємо продукт», тобто чи відповідає він специфікації та вимогам. Валідація відповідає на питання «чи той продукт ми будуємо», тобто чи задовольняє він реальні потреби

користувача. Запам'ятай цю пару: верифікація — відповідність специфікації, валідація — відповідність потребам.

Тестування буває двох основних видів за рівнем знання нутра. Тестування методом білої скриньки враховує внутрішню структуру коду: тестувальник знає, як програма влаштована, і перевіряє конкретні шляхи виконання. Тестування методом чорної скриньки перевіряє програму лише через входи й виходи, не знаючи її внутрішньої будови. Згадай моделі чорної й білої скриньки з попередньої лекції — тут та сама ідея.

Тестування проводять на кількох рівнях, від дрібного до загального. Модульне тестування перевіряє окремі найдрібніші частини, окремі модулі. Інтеграційне тестування перевіряє, чи правильно ці частини працюють разом, у зв'язці. Системне тестування перевіряє всю систему цілком. Валідаційне, або приймальне тестування перевіряє, чи відповідає система очікуванням замовника. Запам'ятай порядок: модульний, інтеграційний, системний, валідаційний — від найменшого до найбільшого.

Окремо варто знати розробку через тестування, англійською Test-Driven Development, скорочено TDD. Її особливість у тому, що спершу пишуть тест, а вже потім код, який цей тест має пройти. Тобто тест випереджає реалізацію. До додаткових технік перевірки якості належать інспекція коду, тобто ручний огляд колегами, перевірка на відповідність стандартам, оцінювання зручності використання й користувацького досвіду, а також перевірка продуктивності та масштабованості.

## **Моделі та методології розробки**

І завершуємо командною роботою та підходами до розробки. Спершу класичні моделі життєвого циклу. Каскадна модель, або водоспадна: етапи виконуються строго послідовно, один за одним, без повернень назад — спочатку всі вимоги, потім усе проєктування, потім уся реалізація. Вона проста, але негнучка. Ітераційна модель: розробку повторюють циклами, поступово уточнюючи й покращуючи продукт на кожній ітерації. Інкрементна модель: систему будують частинами, інкрементами, додаючи готову функціональність шматок за шматком.

Тепер промислові, сучасніші технології розробки. RUP — раціональний уніфікований процес, ітеративний підхід із чітко визначеними фазами. MSF — каркас рішень від компанії Microsoft. Agile — це не конкретна методологія, а ціла філософія гнучкої розробки: цінувати співпрацю, гнучкість і робочий продукт понад жорсткі плани. У межах Agile існують конкретні підходи. Scrum організовує роботу короткими циклами, спринтами, із чіткими ролями: власник продукту, скрам-майстер і команда розробки. Екстремальне програмування, англійською XP, наголошує на технічних практиках: парне програмування, розробка через тестування, постійна інтеграція. Kanban візуалізує роботу на дошці з колонками й обмежує кількість завдань у роботі одночасно, забезпечуючи рівномірний потік.

Наостанок про команду й проєкт. У проєктній команді є різні ролі та обов'язки, і командна робота дає переваги — розподіл навантаження, обмін знаннями, — але несе й ризики: складність комунікації, конфлікти, залежність учасників один від одного. Будь-який ІТ-проєкт проходить основні етапи: планування, виконання, контроль, завершення, — і має життєвий цикл від ідеї до задачі та супроводу.

## Підсумок

Підсумуємо. Проєктування буває структурним, об'єктно-орієнтованим, функціональним, архітектурним та інтерфейсним. UML описує систему діаграмами: класів, послідовностей, комунікації. Патерни — це типові рішення, поділені на породжувальні, структурні й поведінкові. У забезпеченні якості головне — не плутати верифікацію, тобто відповідність специфікації, з валідацією, тобто відповідністю потребам, і знати рівні тестування від модульного до валідаційного. А моделі розробки рухаються від жорсткої каскадної до гнучких Agile-підходів — Scrum, XP, Kanban.

На цьому розділ інженерії ПЗ повністю завершено. Далі за планом — алгоритми та обчислювальна складність.